TRACK 1 · DATA & AI CHALLENGE

Redrob TrustScore

A verified, evidence-based candidate ranking agent

Team **Arcane009**

Team Leader **Abhinav Khatta**

Members **Divyanshu Kapariya · Abhinav Khatta**

Rank the top 100 of 100,000 candidates for a Senior AI Engineer role — without keyword matching, dodging ~80 honeypots, on CPU in under 5 minutes.

github.com/Abhinav09-bits/redrob-trustscore

An autonomous, explainable ranking agent

For each candidate, TrustScore combines seven signals into one auditable score, then returns a ranked top-100 with a plain-language reason for each pick.

Traditional matching

- Embeds the profile, ranks by similarity
- Rewards anyone who stuffs the right keywords
- Walks into honeypots / impossible profiles
- Black box — “the model thought they’re similar”

Redrob TrustScore

- Reads real career evidence — what they actually did
- Penalizes the JD’s stated disqualifiers
- Honeypot kill-switch sinks impossible profiles
- Weights whether the person is even reachable
- Every score is auditable — no black box

What the role needs — and the signals that decide fit

Requirements extracted from the JD

- Production retrieval / embeddings, vector search, ranking & evaluation (NDCG, MRR)
- At product companies — not pure research or services
- 5–9 years; strong Python; Indian metros (Pune, Noida)
- Disqualifiers: title-chasers, consulting-only, research-only, CV/speech-only

Signals that decide fit (beyond keywords)

- Evidence in career text — “built a learning-to-rank model”, “RAG in production”
- Real job title + company type vs. a polished-but-fake skills list
- Behavioral availability: recent login, response rate, open-to-work
- Internal consistency — impossible timelines flag fakes

Seven signals combined into one TrustScore

Role fit	Is the actual role on the AI/ML/search bullseye?	**weight 0.40**
Evidence	Retrieval / ranking / ML work found in career text	**weight 0.32**
Experience	5–9 year sweet spot, decaying outside	**weight 0.16**
Location	India metro / Pune / Noida	**weight 0.12**
Disqualifiers	services-only, job-hopping, research-only, CV-only	**− penalty**
Availability	recent login, response rate, open-to-work	**× 0.55–1.10**
Trust / sanity	honeypot consistency checks	**× kill-switch**

score = (0.40·role + 0.32·evidence + 0.16·exp + 0.12·loc − penalties) × availability

honeypots × 0.001 → they sink below the top 100 · rule-based, no GPU / no LLM at runtime · ties broken by candidate_id

Decisions explained — and kept honest

1

Explained from real data

Each pick gets a 1–2 sentence reason built from the candidate's own fields — title, years, evidence words present, response rate. Honest concerns (notice period, location) are surfaced too.

2

No hallucination

Reasoning is templated from REAL field values, so it cannot invent skills or employers. The score is a transparent sum of named parts — every claim traces to a number.

3

Suspicious / impossible profiles

A honeypot kill-switch flags impossible profiles (tenure > experience, expert skill used 0 months, skill used longer than the career) and pushes them out of the top 100.

QA verified: 0 honeypots and 0 keyword-stuffers in our top 100.

06 · END-TO-END WORKFLOW

From JD input to ranked output

Single command: `python rank.py --candidates ./data/candidates.jsonl --out ./submission`

End-to-end runtime ~17 seconds on CPU, no network.

Layered, dependency-light, reproducible

INPUT LAYER

candidates.jsonl (100K profiles) + Job Description requirements

FEATURE / EVIDENCE EXTRACTION

career-history text · titles · skills · redrob behavioral signals

SCORING ENGINE (rule-based)

role · evidence · experience · location · penalties · availability · honeypot

RANKER

deterministic sort → top 100 → fact-grounded reasoning

OUTPUT + QA

submission.xlsx / .csv · qa.py validates format & traps

CPU-only · no GPU · no network · pure Python + openpyxl · ~17s / 100K · < 16 GB RAM

High-quality ranking, well within compute limits

100,000	~17 s	0	0
candidates ranked	runtime on CPU	honeypots in top 100	keyword-stuffers in top 100

- Top-100 is 100% relevant roles — ML / AI / NLP / Search / Recommendation engineers, no distractor titles
- Manual spot-checks confirm every reason matches the real profile (no hallucination)
- Constraints met with huge margin: 17 s vs 5-min budget, CPU-only, zero network calls, < 16 GB RAM
- Format validated: exactly 100 unique ranks & IDs, all IDs exist, scores non-increasing

Chosen for speed, reproducibility, zero runtime deps

Python 3 (stdlib)	JSON parsing, scoring, CSV output — runtime in seconds, fully reproducible
openpyxl	emits the XLSX output the portal requests
Rule-based scoring	deliberately NO LLM / GPU at runtime → meets 5-min CPU, no-network budget; scales to 200K
Git / GitHub	versioned, reproducible repo with a single reproduce command
Streamlit	hosted sandbox demo for live verification on a small sample

AI tools (Claude) used for code review & drafting — declared honestly; no candidate data sent to any LLM, no LLM in the ranking path.

Everything to reproduce and verify

GitHub repository

github.com/Abhinav09-bits/redrob-trustscore

Live sandbox demo

redrob-trustscore-gba7ntzmq5cwxtkp6ghgkn.streamlit.app

Reproduce command

python rank.py --candidates ./data/candidates.jsonl --out ./submission

Ranked output

submission.xlsx (+ submission.csv) — top 100 with reasoning

QA script

qa.py — verifies valid format and zero traps in the top 100

Metadata

submission_metadata.yaml — team, compute env, honest AI-tools declaration

Thank you

Redrob TrustScore — judging candidates on verified evidence, explainably.

Team Arcane009

github.com/Abhinav09-bits/redrob-truststore

redrob-truststore-gba7ntzmq5cwxtkp6ghgkn.streamlit.app